

Day 6: Functions, Inline & Function Pointers

Platform: nRF52840 | RTOS: Zephyr RTOS

Learning Objectives

Function Declaration vs Definition

- **Declaration** (prototype): tells the compiler the function exists and its signature. Goes in `.h` files.
- **Definition**: the actual implementation. Goes in `.cpp` files.

```
// Declaration (header)
uint32_t clamp_u32(uint32_t val, uint32_t lo, uint32_t hi);

// Definition (source)
uint32_t clamp_u32(uint32_t val, uint32_t lo, uint32_t hi) {
    return (val < lo) ? lo : (val > hi) ? hi : val;
}
```

inline Functions

`inline` is a **hint** to the compiler to copy the function body at every call site, eliminating the call overhead (push/pop registers, branch, return):

```
static inline uint32_t clamp_u32(uint32_t v, uint32_t lo, uint32_t hi) {
    return (v < lo) ? lo : (v > hi) ? hi : v;
}
```

Best used for very small, frequently-called functions. The compiler (`-O2` / `-Os`) will often inline automatically even without the keyword, but `static inline` in a header is the conventional embedded C++ pattern.

Function Pointers

A **function pointer** stores the address of a function. This enables callbacks, dispatch tables, and runtime behavior selection:

```
typedef void (*event_handler_t)(uint8_t event, void *ctx);

void on_button(uint8_t event, void *ctx) { /* ... */ }

event_handler_t handler = on_button;
handler(1, my_ctx); // Calls on_button(1, my_ctx)
```

Callback Pattern for Event-Driven Embedded Code

Embedded systems are **event-driven**: a button press, a timer expiry, a UART byte received. Callbacks let you register a function to call when an event happens:

```
struct event_source {
    event_handler_t callback;
    void            *user_data;
};
```

The `void *user_data` (context pointer) lets the caller pass arbitrary data to the callback without globals.

Zephyr GPIO Callback (struct gpio_callback)

Zephyr's GPIO interrupt system uses a callback struct:

```
static struct gpio_callback my_cb_data;
// Register: link ISR function to a GPIO pin
gpio_init_callback(&my_cb_data, my_isr_func, BIT(button.pin));
gpio_add_callback(button.port, &my_cb_data);
```

Multiple callbacks can be chained on the same GPIO port — Zephyr iterates them all.

static Functions for File-Scope Linkage

Declaring a function `static` at file scope prevents it from being visible in other translation units. This:

- Avoids linker symbol collisions

- Allows the compiler to optimize more aggressively (it knows no one else calls it)
 - Is the embedded C++ convention for module-private functions
-

Practice Tasks

1. Write a dispatch table: array of 4 function pointers, call each based on a command byte
 2. Implement a debounce callback that ignores button events within 50ms of the last one
 3. Replace the Zephyr GPIO callback with a C++ class that stores the callback and context
 4. Use `__attribute__((noinline))` on a function — verify in disassembly that it's never inlined
-

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day06
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day06/main.cpp](#)